

Software & Architecture Types

Full lecture notes on the classification of software, the operating system, the toolchain that turns source code into a running program, the two hardware models — von Neumann and Harvard — and the instruction pipeline they support.

Presented by **Dr. Surani Tissera** Department of Computer Science University of Sri Jayewardenepura



U1 INTRODUCTION

What is software?

A software, or program, is a set of instructions written to solve a specific problem. That definition is broad on purpose — it covers everything from a five-line script to an entire operating system.

Sort a set of numbers

Find roots of a quadratic equation

Find the inverse of a matrix

C compiler → machine language

Editor for creating documents

Operating system for using the computer

Application software vs. system software

Broadly, every program falls into one of two categories, distinguished by *who* it ultimately serves.

APPLICATION SOFTWARE

Helps the user solve a particular user-level problem. It may need system software in order to actually execute, and in most cases it resides on the computer's hard disk or on removable storage media (DVD, USB drive, etc.).

- Financial accounting package
- Mathematical packages — MATLAB, Mathematica
- An app to book a cab
- An app to monitor the health of a person

SYSTEM SOFTWARE

A collection of programs that helps users create, analyze, and run their programs — it manages the machine itself. Typical operations it carries out:

- Handling user requests
- Managing application programs and storing them as files
- File management in secondary storage devices
- Running standard applications — word processor, browser
- Managing I/O units
- Program translation from source form to object form
- Linking and running user programs

Some of the most commonly used system software: the **operating system** (Windows, Linux, macOS, Android), **compilers and assemblers, linkers and loaders**, and **editors and debuggers**. These are explored below because together they form the toolchain a programmer relies on every day.

From source code to a running program

Six tools work together to take a program from an idea to something the CPU can execute. Each does one job, then hands off to the next.



COMPILERS AND ASSEMBLERS

Compiler — translates a high-level language program (C, C++, Java) into machine code that the processor can execute directly. The C compiler mentioned in U1 does exactly this. **Assembler** — translates assembly language, a low-level form close to machine instructions, into binary machine code. Both perform *program translation* — converting code from source form to object form — just at different levels of abstraction.

LINKERS AND LOADERS

Linker — after compiling, a program often isn't complete on its own: it may call functions from libraries (like `printf`) or reference code compiled in other files. The linker combines multiple object files and libraries into a single executable, resolving references between them — matching a function call to where that function actually

lives. **Loader** — takes the finished executable from disk and loads it into main memory, sets up the necessary memory addresses, and hands control to the OS to begin execution.

ANALOGY If compiling is writing individual chapters of a book, the linker binds all the chapters together into one book, and the loader is what puts that finished book on your desk so you can actually start reading — executing — it.

EDITORS AND DEBUGGERS

Editor — a program that lets you create and modify text or code, such as writing source code or a document.

Debugger — helps find and fix errors while a program is running: pause execution with breakpoints, step through line by line, inspect variable values, and trace what's going wrong instead of guessing blindly.

WHY THEY'RE ALL "SYSTEM SOFTWARE"

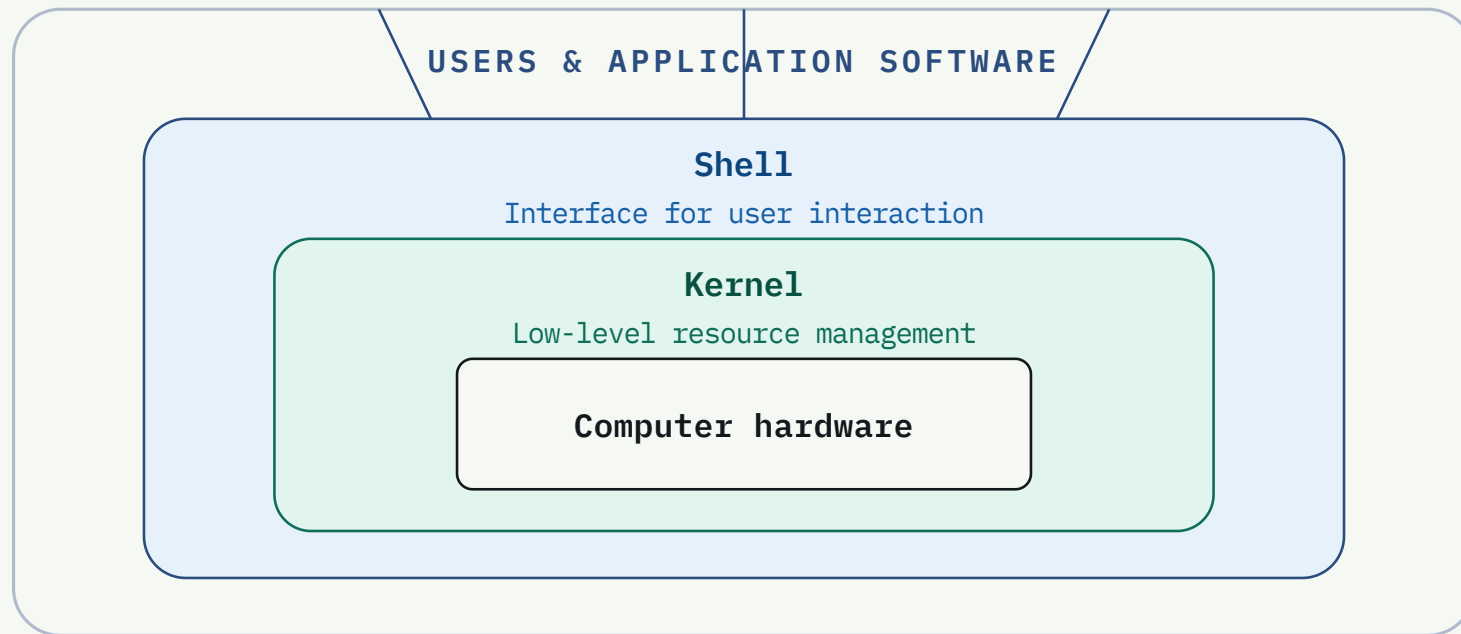
None of these six tools solve an end user's problem directly, the way an accounting app would. Instead they exist to help build, prepare, and manage *other* programs — which is exactly the definition of system software from U2.



U4 OPERATING SYSTEM

The interface between hardware and user

An operating system provides an interface between computer hardware and users. It is structured in two layers, and it is an instance of a program that never terminates — it keeps running until the machine is switched off or the user manually shuts it down.



Kernel – contains low-level routines for resource management • Shell – provides an interface for users to interact with hardware through the kernel

The OS goal shifts with the system it runs on

Depending on the intended use of the computer system, the objective the OS optimizes for is different. Four goals, each with real-world examples and a way to feel the idea:

CLASSICAL MULTIPROGRAMMING SYSTEMS

Several user programs are loaded in memory at once. The system switches to another program whenever the running one gets blocked waiting on I/O. **Goal: maximize resource utilization.**

IBM OS/360 mainframes

University batch computing centers, 1970s

ANALOGY A chef cooking multiple dishes at once — while dish A bakes in the oven (waiting on I/O), the chef chops vegetables for dish B (CPU keeps working) instead of standing idle watching the oven.

MODERN TIME-SHARING SYSTEMS

Widely used because every user can now afford a separate terminal. Processor time is shared among a number of interactive users. **Goal: minimize user response time.**

Desktop OS multitasking — browser, editor, music player

A university lab where many students share one server

ANALOGY A waiter serving many tables — they don't finish one table fully before starting the next; they take quick turns so every table feels attended to promptly.

REAL-TIME SYSTEMS

Several applications run with specific deadlines, which can be hard or soft. Operation is interrupt-driven — the processor is interrupted when a task arrives.

Missile control system (hard)

Airbag deployment (hard)

Video streaming decode (soft)

Patient ICU heart monitor

Automotive control system

Industrial robotic arm

Hard deadline: missing it means failure — an airbag firing late is unacceptable. Soft deadline: missing it just degrades quality — a late video frame causes a stutter, not a disaster.

MOBILE (PHONE) SYSTEMS

User responsiveness is the most important factor. A program that makes the system slow or hogs too much memory may be forcibly stopped.

Android/iOS killing a background app so the camera opens instantly

ANALOGY A restaurant host who tells a lingering, non-paying customer to free up a table so new paying guests aren't kept waiting — the system sacrifices something less important (background app) to keep the active experience (foreground app) snappy.

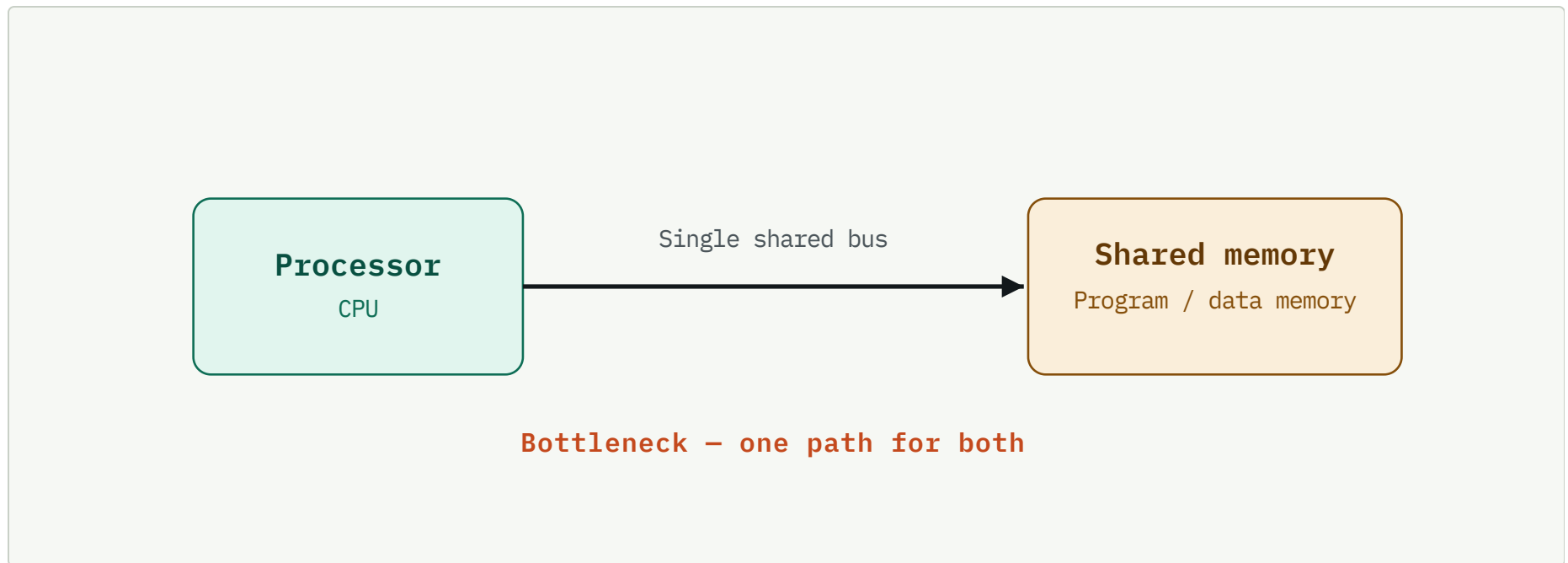
How the hardware organizes memory

Underneath any OS, the processor needs two things constantly: instructions to run, and data to run them on. How those are stored and reached defines the computer's architecture — broadly classified into two types.

VON NEUMANN ARCHITECTURE

Named after mathematician **John von Neumann**, this is the traditional model used in most general-purpose computers. Instructions and data are stored in the **same memory module**, reached through a single shared bus.

In practice: the CPU sends an address on the bus to fetch an instruction, then later sends another address on the *same* bus to fetch or store data. Because there is only one bus, these requests happen one at a time, in sequence.

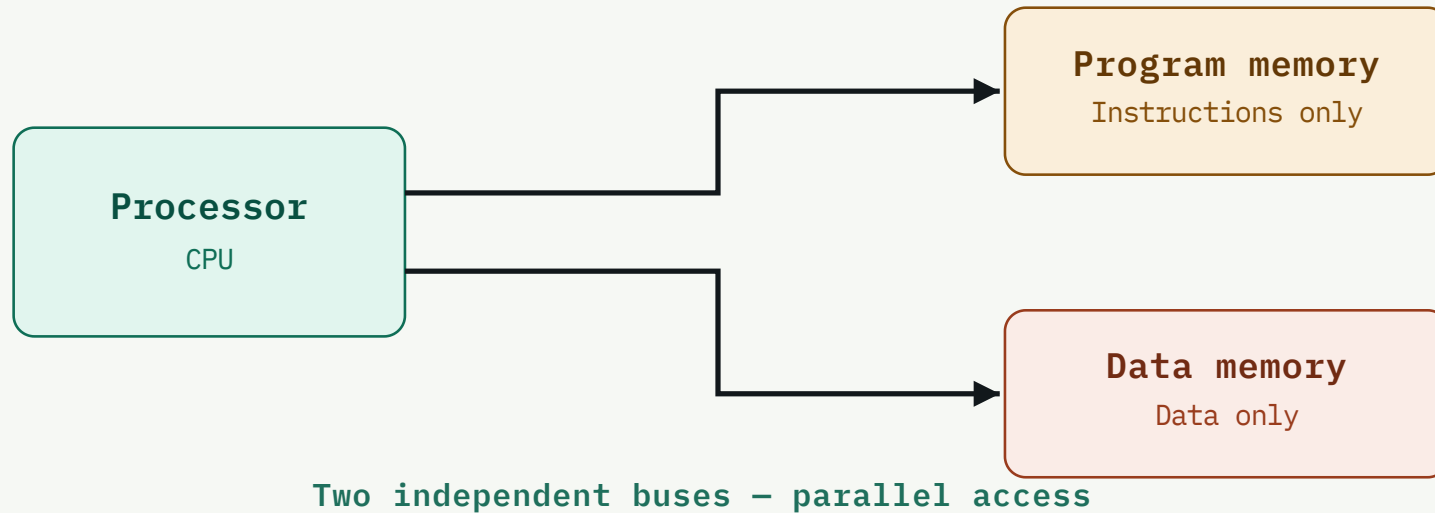


The core problem — the "von Neumann bottleneck": since instructions and data compete for the same bus, the CPU can be fast, but the connection between CPU and memory acts like a narrow pipe throttling how quickly data and instructions can flow, even if the processor itself could do more work per cycle. All instructions and data are moved back and forth through that one pipe.

Why it's still popular: it is more flexible and easier to implement — the same memory can be dynamically allocated between code and data as needed — which is why it remains suitable for most general-purpose processors.

HARVARD ARCHITECTURE

Named after the **Harvard Mark I** computer, this model physically separates program memory and data memory, each with its own dedicated bus to the processor. Instructions are stored in program memory, and data are stored in data memory — and because they're separate, instruction and data accesses can be done **in parallel**.



Where it's used: many microcontrollers and pipelines with separate instruction and data caches follow this concept. Even CPUs whose main memory is Von Neumann-style often use separate L1 instruction and data caches internally – sometimes called a *modified Harvard architecture*. **Remaining limitation:** the processor-memory bottleneck remains – each bus can still individually bottleneck if traffic on it is heavy, it just removes the *shared* conflict between instructions and data.

ARCHITECTURE	MEMORY LAYOUT	BOTTLENECK	TYPICAL USE
Von Neumann	Instructions and data share one memory module and one bus.	Shared bus contention — fetch and access compete for the same path	General-purpose processors (desktops, servers)
Harvard	Separate program memory and data memory, each with its own bus.	Each bus can still bottleneck individually, but no cross-conflict	Microcontrollers; L1 instruction/data caches in modern CPUs

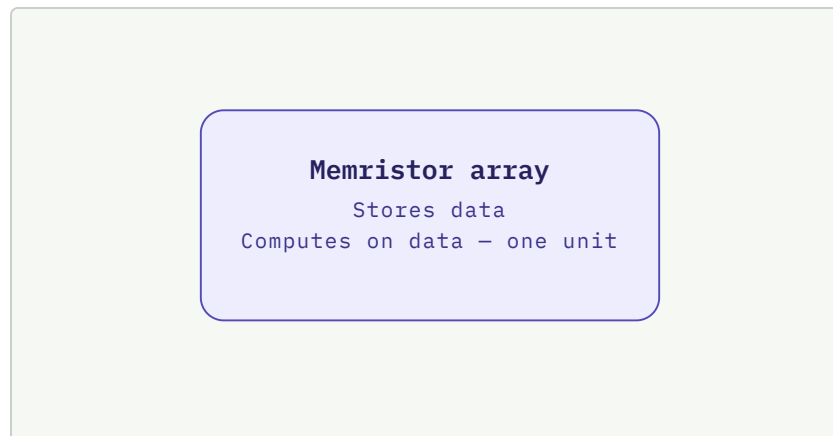
WHY HARVARD HELPS PIPELINING

In clock cycle 4, instruction 4 is trying to fetch an instruction (IF) while instruction 1 may be trying to access data (MEM) — both happening in the very same clock cycle. On von Neumann hardware, one of these two operations has to wait, causing a pipeline slowdown. On Harvard hardware, the operations go on without any speed penalty, since instruction and data memories are physically separate.

In-memory computing and memristors

Both models above share one assumption: computation happens in the processor, storage happens in memory, and data must travel between them every single time it's needed. That back-and-forth movement costs time and energy — especially as datasets grow huge, as with AI and big-data workloads. Several architectural concepts have been proposed that deviate significantly from both the von Neumann and Harvard model.

In-memory computing challenges the assumption directly: what if storage and computation could be done in the *same* functional unit?



WHAT'S A MEMRISTOR?

A **memristor** ("memory resistor") is an electronic component whose resistance changes based on the history of current that has passed through it — and it *remembers* that resistance value even after power is removed (non-volatile). This is fundamentally different from a transistor, the building block of traditional processors and memory chips.

WHY THIS MATTERS

- **Storage** — memristors can be arranged into high-capacity, non-volatile resistive memory systems, meaning data survives without power, unlike regular RAM.
- **Computation** — because a memristor's resistance can be set and read, arrays of them can be wired so that reading them out performs a computation directly, such as matrix multiplication, which is heavily used in neural networks. The computation happens as a physical side effect of accessing the memory, not as a separate step afterward.

BIG PICTURE SIGNIFICANCE

This directly attacks the processor-memory bottleneck that both Von Neumann and Harvard architectures still suffer from — instead of shipping data back and forth between a compute unit and a storage unit, the same physical unit does both. It's especially promising for AI workloads, which involve massive, repetitive matrix operations where the traditional back-and-forth data movement is a major energy and speed cost.

QUICK COMPARISON SUMMARY

ARCHITECTURE	INSTRUCTION/DATA STORAGE	BUS(ES)	BOTTLENECK	TYPICAL USE
Von Neumann	Same memory	1 shared	Shared bus contention	General-purpose CPUs
Harvard	Separate memories	2 independent	Per-bus, no cross-conflict	Microcontrollers, CPU caches
In-memory computing	Unified (compute = storage)	None needed	Removes it almost entirely	Emerging, memristor-based, AI-focused

Overlapping instruction execution

Instruction execution is typically divided into five stages. If every instruction had to fully complete all five stages before the next could even start, the CPU would be very slow — one instruction at a time, start to finish. A pipeline architecture overlaps these five stages across instructions instead, for a significant speedup.

Trace a single instruction through all five stages using an example: `ADD R1, R2, R3` — meaning $R1 = R2 + R3$.

IF — INSTRUCTION FETCH

The CPU retrieves the next instruction from memory (or instruction cache) using the address held in the **program counter (PC)**. The instruction is fetched from instruction memory, then the PC is incremented to point to the next instruction, ready for the following fetch.

ANALOGY Picking up the next recipe card from a stack before you start cooking.

ID — INSTRUCTION DECODE

The fetched instruction (just a string of bits) is decoded to figure out the operation to perform and which registers are involved (R1, R2, R3). The control unit generates the control signals, and the source registers (R2 and R3) are read from the register file.

ANALOGY Reading the recipe card and figuring out which ingredients — registers — you need to grab from the pantry.

EX – EXECUTE (ALU OPERATION)

The actual computation happens in the **ALU**. For **ADD**, it adds R2 and R3. For memory instructions (LOAD/STORE), the ALU computes the memory address. For branch instructions, it may evaluate a condition and calculate the target address.

ANALOGY The actual cooking step — mixing the ingredients together.

MEM – MEMORY ACCESS

If the instruction needs to interact with data memory, it happens here. A LOAD reads data from the computed address; a STORE writes data to it. For our ADD example, this stage does nothing — ADD doesn't touch memory, so the result just passes through.

ANALOGY Going to the fridge to grab or put away an item — but if the recipe step doesn't need the fridge, you skip it and move on.

WB – WRITE BACK

The final result is written back into the register file. For our example, the sum of R2 + R3 is written into R1. For a LOAD instruction, the data fetched from memory in the MEM stage is written into the destination register.

ANALOGY Plating the finished dish and putting it on the table — the result is now available for use.

WHY SPLIT INTO FIVE STAGES?

Because each stage uses **different hardware**, they can all run simultaneously for different instructions — while instruction 1 is in MEM, instruction 2 can already be in EX, instruction 3 in ID, and instruction 4 in IF, all in the same clock cycle.

STAGE	HARDWARE USED
IF	Instruction memory / cache, program counter
ID	Register file (read), decode logic
EX	ALU
MEM	Data memory / cache
WB	Register file (write)

ANALOGY Five chefs in a kitchen, each handling one stage of the recipe for a different order at the same time, instead of one chef doing everything for one order before starting the next.

Basic 5-stage pipelining diagram

Instr.	1	2	3	4	5	6	7
1	IF	ID	EX	MEM	WB		
2		IF	ID	EX	MEM	WB	
3			IF	ID	EX	MEM	WB
4				IF	ID	EX	MEM
5					IF	ID	EX

Instruction fetch
 Instruction decode
 Execute (ALU)
 Memory access
 Write back

CONNECTING BACK TO HARVARD VS. VON NEUMANN

Look at clock cycle 4: instruction 4 needs to fetch (IF) while instruction 1 needs to access memory (MEM) — both memory operations at once. In **Von Neumann** architecture (single shared memory/bus), these collide and one must stall, slowing the pipeline. In **Harvard** architecture (separate instruction and data memories), both proceed simultaneously with no conflict, since they use different memory paths. This is the key practical payoff of Harvard architecture — it removes a major source of pipeline stalls.

NEXT LECTURE

Instruction Set Architecture

This is a property of the Department of Computer Science, Faculty of Applied Science, University of Sri Jayewardenepura.
Notes compiled from ICT 1022.2.0 – Computer System Architecture, presented by Dr. Surani Tissera.